



School of Future Tech

Case Study Report

High-Frequency Trading (HFT) Price Lookup Engine

BY

SAHIL GHONE (150096725002)

RAAD DALWAI (150096725207)

NANDINI SOLANKI(150096725189)

AARYAN KUCHEKAR(150096725048)

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

High-Frequency Trading (HFT) systems operate in extremely fast-paced financial environments where even microseconds matter. These systems process massive volumes of stock price data and require instant decision-making.

This case study focuses on building a **Price Lookup Engine** that efficiently finds the **closest stock price less than or equal to a target value**. Such functionality is critical for traders who need to execute trades at optimal prices without delay.

2. Problem Statement / Case Background (Abstract)

In modern trading platforms, stock prices are updated continuously, generating large time-series datasets. Traders often need to find the most recent price that does not exceed a specified target.

Challenges include:

- Handling large datasets efficiently
- Ensuring ultra-fast lookup times
- Maintaining accuracy in real-time scenarios

Traditional search methods are too slow, making it necessary to adopt optimized algorithms.

3. Problem Statement / Case Study Design

Objective:

To design a system that finds the **closest stock price $\leq$ target price** in the fastest possible time.

Inputs:

- A list of stock prices with timestamps
- A target price entered by the user

Outputs:

- Closest price less than or equal to target
- Corresponding timestamp

Constraints:

- Data must be sorted before searching
- Must handle cases where no valid price exists

System Flow:

1. Store price data
2. Sort the dataset
3. Accept target input
4. Perform optimized search
5. Display result

4. Methods & Algorithms / Technology Applied

Algorithm Used: Binary Search (Lower Bound Concept)

Binary Search is used because:

- It divides the dataset into halves repeatedly
- It reduces search time significantly

Time Complexity

- $O(\log n) \rightarrow$ Extremely fast

Key Logic

- If mid value $\leq$ target $\rightarrow$ move right
- If mid value $>$ target $\rightarrow$ move left
- Track the best possible answer

Technologies Used

- Programming Language: C++
- Data Structures: `vector`, `struct`
- Standard Libraries: `<iostream>`, `<vector>`, `<algorithm>`

5. Implementation Details and Snapshots

Data Structure Used

```
struct PricePoint {  
  
    double price;  
  
    long long timestamp;  
  
};
```

Core Function

- `findClosestPrice()` performs binary search
- Returns closest valid price

hft.cpp ×

hft.cpp > findClosestPrice(vector<PricePoint>&, double)

```
1  #include<iostream>
2  #include<vector>
3  using namespace std;
4
5  struct PricePoint {
6      double price;
7      long long timestamp;
8  };
9
10
11  PricePoint findClosestPrice(vector<PricePoint>& data, double target){
12
13      if(data.empty()){
14          return PricePoint{0.0, -1};
15      }
16
17      int i = 0;
18      int j = data.size() - 1;
19      int upperBoundIndex = j + 1;
20      while(i<=j){
21          int midIndex = i + (j-i)/2;
22          if(data[midIndex].price > target){
23              upperBoundIndex =midIndex;
24              j = midIndex-1;
25          }
26          else if(data[midIndex].price <= target){
27              i = midIndex+1;
28          }
29      }
30
31      return upperBoundIndex==0? PricePoint{0.0, -1}:data[upperBoundIndex-1];
32  }
33
```

```

34 int main(){
35     vector<PricePoint> data = {
36         {2100.50, 1622548800000},
37         {5420.75, 1622548801000},
38         {8900.00, 1622548802000},
39         {12450.25, 1622548803000},
40         {18700.60, 1622548804000},
41         {22100.10, 1622548805000},
42         {28500.00, 1622548806000},
43         {1250.45, 1622548807000}
44     };
45     double targetPrice;
46
47     sort(data.begin(), data.end(), [](const PricePoint& a, const PricePoint& b) {
48         return a.price < b.price;
49     });
50
51
52     cout << "=====" << endl;
53     cout << "    HFT PRICE LOOKUP ENGINE (INR)    " << endl;
54     cout << "=====" << endl;
55     cout << "Enter target price (INR): ";
56     cin >> targetPrice;
57
58
59
60     PricePoint closest = findClosestPrice(data, targetPrice);
61
62
63     if (closest.timestamp != -1) {
64         cout << "\n[RESULT FOUND]" << endl;
65         cout << "● Nearest Price   : ₹ " << closest.price << endl;
66         cout << "● Timestamp       : " << closest.timestamp << " ns" << endl;
67     } else {
68         cout << "\n[NO MATCH] No price available <= ₹ " << targetPrice << endl;
69     }
70     cout << "=====" << endl;
71
72     return 0;
73 }

```

Steps in Code

1. Check if data is empty
2. Apply binary search
3. Track upper bound index
4. Return closest value

Sample Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● nandinisolanki10@Nandinis-MacBook-Air DSA_MINIPROJECT % ./hft
=====
HFT PRICE LOOKUP ENGINE (INR)
=====
Enter target price (INR): 3456

[RESULT FOUND]
● Nearest Price : ₹ 2100.5
● Timestamp    : 1622548800000 ns
=====
○ nandinisolanki10@Nandinis-MacBook-Air DSA_MINIPROJECT %
```


6. Results and Conclusion

Results

- Fast and efficient price lookup
- Accurate nearest price detection
- Handles edge cases (no match)

Performance

- Achieves **logarithmic time complexity**
- Suitable for large datasets

Conclusion

This system demonstrates how **Binary Search** can optimize financial data processing. It provides a scalable and efficient solution for HFT environments where speed is critical.

Future Improvements:

- Real-time data streaming integration
- Multi-stock support
- Integration with trading APIs
- Visualization dashboard

7. References

- C++ Standard Library Documentation
- Data Structures and Algorithms textbooks
- Online resources on Binary Search and HFT systems
- Financial market basics and trading systems